

# Neural Machine Translation By Jointly Learning To Align And Translate

이 논문은 입력 문장 전체를 하나의 고정 길이 벡터로 압축하던 기존 RNN Encoder-Decoder의 정보 병목을 해결하기 위해, 번역 단어를 생성할 때마다 입력 문장의 관련 부분을 선택적으로 참고하는 Attention mechanism을 제안한다.

## 서론

### 연구 배경과 문제의식

기계번역은 한 언어로 작성된 문장을 다른 언어의 문장으로 변환하는 문제이다. 전통적인 phrase-based 통계적 기계번역 시스템은 여러 하위 구성 요소를 별도로 설계하고 각각 조정해야 했다. 반면 Neural Machine Translation은 하나의 신경망이 입력 문장을 읽고 번역 문장을 생성하도록 전체 과정을 end-to-end로 학습하는 접근이다.

당시 대표적인 Neural Machine Translation 구조는 RNN 기반의 Encoder-Decoder였다. Encoder는 길이가 가변적인 입력 문장을 하나의 고정 길이 벡터로 변환하고, Decoder는 이 벡터를 바탕으로 번역 문장을 순차적으로 생성한다. 모델은 주어진 원문  $x$ 에 대해 올바른 번역문  $y$ 의 조건부 확률  $p(y | x)$ 를 최대화하도록 학습된다.

이 구조는 여러 하위 구성 요소를 따로 설계하지 않고 하나의 신경망으로 번역을 수행할 수 있다는 장점이 있다. 그러나 입력 문장의 길이에 관계없이 Encoder가 문장 전체의 정보를 항상 동일한 크기의 벡터에 압축해야 한다는 문제가 존재한다. 문장이 짧을 때는 비교적 충분한 정보를 담을 수 있지만, 문장이 길어질수록 앞부분의 정보나 세부 표현이 손실될 가능성이 커진다. 논문은 Cho et al. (2014b)의 결과를 근거로 기존 Encoder-Decoder가 입력 문장이 길어질수록 성능이 빠르게 저하된다는 점을 지적한다.

기존 Encoder-Decoder에서는 Encoder가 입력 문장 전체를 하나의 context vector  $c$ 로 압축한다.

$$x_1, x_2, \dots, x_{T_x} \longrightarrow h_1, h_2, \dots, h_{T_x} \longrightarrow c$$

Decoder는 번역문의 모든 단어를 생성할 때 동일한 context vector  $c$ 를 사용한다.

$$p(y_i | y_1, \dots, y_{i-1}, c)$$

따라서 Encoder가 입력 문장의 일부 정보를 context vector에 충분히 보존하지 못하면 Decoder는 이후에 원문의 해당 부분을 다시 확인할 수 없다. 이처럼 입력 문장의 모든 정보를 하나의 고정 길이 표현을 통해 전달해야 한다는 제약을 **fixed-length bottleneck**이라고 볼 수 있다.

이를 직관적으로 비유하면, 짧은 문장을 한 줄로 요약하는 것은 가능하지만 매우 긴 문장의 모든 세부 정보를 똑같이 한 줄에 담기는 어려운 것과 같다. 저자들은 이러한 고정 길이 벡터가 기존 Encoder-Decoder의 성능을 제한하는 핵심 원인이라고 보았다.

### 논문의 핵심 아이디어

저자들은 문장 전체를 하나의 벡터로 압축하는 대신, 입력 문장의 각 위치에 대한 표현을 모두 유지하도록 모델을 변경하였다. 그리고 Decoder가 번역 단어를 생성할 때마다 입력 문장의 어느 위치가 현재 단어와 관련 있는지를 계산해 필요한 정보를 선택적으로 가져오도록 하였다.

기존 모델이 모든 출력 시점에서 동일한 context vector를 사용했다면, 제안 모델은 출력 단어마다 서로 다른 context vector를 사용한다.

$$c \longrightarrow c_1, c_2, \dots, c_{T_y}$$

이 과정에서 계산되는 입력 위치와 출력 단어 사이의 가중치가 **soft alignment**이다. 모델은 별도의 단어 정렬 정답을 제공받지 않아도 번역 성능을 높이는 방향으로 정렬 관계를 함께 학습한다. 이것이 논문 제목의 "jointly learning to align and translate"가 의미하는 바이다.

## 본론

### 기존 RNN Encoder-Decoder

기존 Encoder-Decoder에서 Encoder RNN은 입력 단어를 순서대로 읽으며 hidden state를 갱신한다.

$$h_t = f(x_t, h_{t-1})$$

입력 문장을 모두 읽은 뒤 hidden state들의 정보를 이용해 하나의 context vector를 만든다.

$$c = q(\{h_1, \dots, h_{T_x}\})$$

대표적으로 Sutskever et al. (2014)은 마지막 hidden state  $h_{T_x}$ 를 context vector로 사용하였다.

Decoder는 이전까지 생성한 단어와 context vector를 바탕으로 번역문 전체의 확률을 다음과 같이 분해한다.

$$p(y | x) = \prod_{t=1}^{T_y} p(y_t | y_1, \dots, y_{t-1}, c)$$

이 구조에서는 번역문의 주어, 동사, 목적어를 생성할 때 각각 필요한 원문의 정보가 다르더라도 모든 출력 단어가 동일한 context vector만 참고해야 한다. 입력 문장이 길어질수록 하나의 벡터가 보존해야 하는 정보가 많아지므로 정보 병목이 심해진다.

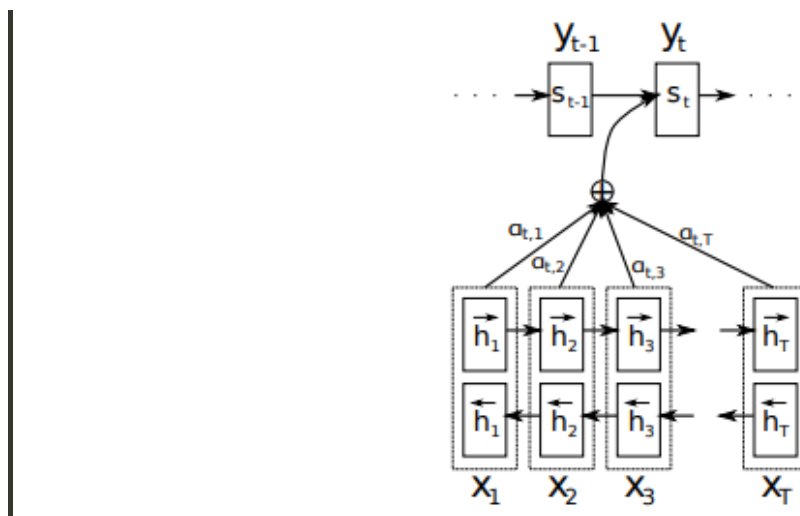
### 제안 모델: Attention 기반 Encoder-Decoder

본 논문의 모델은 크게 Bidirectional RNN Encoder, alignment model, Attention을 사용하는 Decoder로 구성된다.

Encoder는 입력 문장의 각 위치에 대응하는 annotation을 생성한다. Decoder는 번역 단어를 하나 생성할 때마다 각 annotation과 이전 Decoder hidden state 사이의 관련성을 계산한다. 이후 이 관련성을 정규화한 Attention weight를 이용해 현재 출력 시점에 필요한 context vector를 만든다.

전체 흐름은 다음과 같다.

입력 단어 → BiRNN annotations → alignment scores → Attention weights → context vector → 출력 단어



**Figure 1.** 제안 모델의 전체 구조. Decoder가  $t$ 번째 번역 단어  $y_t$ 를 생성할 때 입력 문장의 각 annotation  $h_j$ 에 가중치  $\alpha_{t,j}$ 를 부여하고, 이를 결합해 현재 출력 시점의 context vector를 구성한다.

### Bidirectional RNN Encoder

일반적인 단방향 RNN은 문장을 앞에서 뒤로 읽기 때문에 특정 위치의 hidden state는 현재 단어와 이전 단어들의 정보를 주로 포함한다. 그러나 어떤 단어의 의미를 정확히 표현하려면 앞쪽 문맥뿐 아니라 뒤쪽 문맥도 함께 고려해야 할 수 있다.

이를 위해 저자들은 forward RNN과 backward RNN으로 구성된 Bidirectional RNN을 사용한다. Forward RNN은 문장을 앞에서 뒤로 읽고, backward RNN은 문장을 뒤에서 앞으로 읽는다.

$$\vec{h}_j = \vec{f}(x_j, \vec{h}_{j-1}), \quad \overleftarrow{h}_j = \overleftarrow{f}(x_j, \overleftarrow{h}_{j+1})$$

각 입력 위치  $j$ 의 annotation  $h_j$ 는 두 방향의 hidden state를 연결하여 만든다.

$$h_j = \begin{bmatrix} \overrightarrow{h_j} \\ \overleftarrow{h_j} \end{bmatrix}$$

따라서  $h_j$ 는  $j$ 번째 단어 이전과 이후의 문맥을 모두 반영한다. Encoder의 최종 출력은 하나의 벡터가 아니라 입력 문장의 각 위치에 대응하는 annotation sequence가 된다.

$$(h_1, h_2, \dots, h_{T_x})$$

논문은 RNN이 비교적 최근의 입력을 더 잘 표현하는 경향이 있으므로, 각 annotation이 문장 전체의 정보를 어느 정도 포함하면서도 해당 위치 주변의 정보에 특히 초점을 맞춘 표현이 된다고 설명한다.

## Alignment score와 Attention weight

Decoder가  $i$ 번째 번역 단어를 생성한다고 하자. 먼저 이전 Decoder hidden state  $s_{i-1}$ 과 입력 위치  $j$ 의 annotation  $h_j$  사이의 관련성을 계산한다.

$$e_{ij} = a(s_{i-1}, h_j)$$

여기서  $a$ 는 feedforward neural network로 구현된 alignment model이다.  $e_{ij}$ 는 현재 출력 위치  $i$ 와 입력 문장의  $j$ 번째 위치 주변이 얼마나 잘 대응하는지를 나타내는 점수이다.

현재 Decoder 상태  $s_i$ 가 아니라 이전 상태  $s_{i-1}$ 을 사용한다는 점이 중요하다. 현재 상태는 다음과 같이 context vector  $c_i$ 를 이용해 계산된다.

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

그런데  $c_i$ 를 만들려면 먼저 alignment score와 Attention weight를 계산해야 한다. 따라서 alignment score에 아직 계산되지 않은  $s_i$ 를 사용하면 순환 참조가 발생한다. 실제 계산 순서는 다음과 같다.

$$s_{i-1} \rightarrow e_{ij} \rightarrow \alpha_{ij} \rightarrow c_i \rightarrow s_i$$

Alignment score에 softmax를 적용하면 Attention weight  $\alpha_{ij}$ 를 얻는다.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

각 출력 시점  $i$ 에 대해 모든 Attention weight의 합은 1이다.

$$\sum_{j=1}^{T_x} \alpha_{ij} = 1$$

$\alpha_{ij}$ 가 크다는 것은  $i$ 번째 번역 단어를 생성할 때 입력 문장의  $j$ 번째 위치를 중요하게 참고했다는 의미이다.

논문에서는 alignment model을 tanh 비선형성을 사용하는 작은 feedforward neural network로 구현한다.

$$a(s_{i-1}, h_j) = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$$

두 벡터를 각각 선형 변환한 뒤 더하고, tanh를 통과시킨 결과를 하나의 점수로 변환하는 additive 형태의 함수이다.

## 동적으로 계산되는 context vector

현재 출력 시점의 context vector  $c_i$ 는 입력 annotation들의 가중합으로 계산된다.

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

기존 모델에서는 하나의 고정된  $c$ 를 모든 출력 단어가 공유했지만, 제안 모델에서는 출력 시점마다 Attention weight를 다시 계산하므로 서로 다른  $c_i$ 가 만들어진다.

논문은 이 과정을 expected annotation을 계산하는 것으로도 해석한다.  $\alpha_{ij}$ 를 출력 단어  $y_i$ 가 입력 위치  $j$ 와 정렬될 가능성으로 보면,  $c_i$ 는 가능한 정렬 관계 전체를 고려한 annotation의 기대값이다.

하나의 입력 위치만 선택하는 hard alignment와 달리, soft alignment는 모든 입력 위치에 연속적인 가중치를 부여한다. 따라서 하나의 출력 단어를 생성하기 위해 여러 입력 단어의 정보가 필요한 경우에도 자연스럽게 대응할 수 있다. 또한 길이가 서로 다른 원문 구와 번역 구를 정렬할 때 일부 단어를 인위적으로 [NULL]에 대응시키지 않아도 된다는 장점이 있다.

## Attention Decoder

Decoder의 hidden state는 이전 hidden state, 이전에 생성한 단어, 현재 시점의 context vector를 이용해 갱신된다.

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

다음 단어의 조건부 확률은 다음과 같이 계산된다.

$$p(y_i | y_1, \dots, y_{i-1}, x) = g(y_{i-1}, s_i, c_i)$$

Decoder는 번역 단어를 하나 생성할 때마다 다음 과정을 반복한다.

1. 이전 Decoder hidden state와 각 입력 annotation 사이의 alignment score를 계산한다.
2. Score에 softmax를 적용해 Attention weight를 구한다.
3. Annotation들의 가중합으로 현재 context vector를 만든다.
4. Context vector를 이용해 Decoder 상태를 갱신하고 다음 단어를 생성한다.

이 구조를 통해 입력 문장의 정보가 하나의 고정 벡터에 갇히지 않고, Decoder가 필요한 시점에 관련된 입력 위치의 정보를 선택적으로 가져올 수 있다. 논문에서는 이 과정을 Decoder가 입력 문장의 관련 부분을 soft-search하는 것으로 설명한다.

## 정렬과 번역의 공동 학습

본 논문에서는 alignment를 별도로 추론해야 하는 latent variable로 취급하지 않는다. Alignment model이 미분 가능한 soft alignment를 직접 계산하기 때문에 번역 목적함수의 gradient가 alignment model을 거쳐 Encoder까지 역전파될 수 있다.

Softmax와 가중합 연산이 모두 미분 가능하므로 Decoder, alignment model, Encoder는 올바른 번역문의 log-probability를 높이는 하나의 목적함수 아래에서 함께 학습된다. 따라서 별도의 단어 정렬 label이 없어도 모델은 번역 성능을 높이는 데 유용한 정렬 패턴을 스스로 학습한다.

즉, alignment를 별도의 잠재변수로 두어 추론하는 대신 모델 내부에서 연속적인 가중치로 직접 계산하고, 번역 모델과 end-to-end로 공동 학습한다는 점이 본 논문의 핵심적인 기여이다.

## 실험 설정

저자들은 WMT '14 영어-프랑스어 병렬 말뭉치를 사용해 모델을 평가하였다. 전체 말뭉치는 약 8억 5,000만 단어로 구성되어 있으며, Axelrod et al. (2011)의 데이터 선택 방법을 적용해 약 3억 4,800만 단어 규모의 학습 데이터를 구성하였다.

news-test-2012와 news-test-2013을 이어 붙여 validation set으로 사용하고, 학습 데이터에 포함되지 않은 3,003개 문장으로 구성된 news-test-2014를 test set으로 사용하였다.

영어와 프랑스어에서 각각 빈도가 높은 30,000개의 단어만 vocabulary에 포함하고, vocabulary에 없는 단어는 [UNK]로 변환하였다. Lowercasing이나 stemming과 같은 추가적인 전처리는 적용하지 않았다.

실험에서는 다음 두 종류의 모델을 비교하였다.

- **RNNencdec:** 하나의 고정 길이 context vector를 사용하는 기존 Encoder-Decoder
- **RNNsearch:** Attention과 soft alignment를 사용하는 제안 모델

각 모델은 학습에 포함하는 문장의 최대 길이에 따라 다시 구분된다.

- **-30:** 길이가 최대 30단어인 문장으로 학습
- **-50:** 길이가 최대 50단어인 문장으로 학습

두 모델 모두 gated hidden unit을 사용하였다. Minibatch의 크기는 80이었으며, SGD와 Adadelta를 이용해 각 모델을 약 5일간 학습하였다. 번역문 생성에는 beam search를 사용했고, 성능은 BLEU score로 평가하였다. 전통적인 phrase-based 번역 시스템인 Moses와도 성능을 비교하였다.

## 실험 결과

### 정량적 성능 비교

모든 조건에서 제안 모델인 RNNsearch가 기존 RNNencdec보다 높은 BLEU score를 기록하였다.

| Model         | 전체 문장 | [UNK]가 없는 문장 |
|---------------|-------|--------------|
| RNNencdec-30  | 13.93 | 24.19        |
| RNNsearch-30  | 21.50 | 31.44        |
| RNNencdec-50  | 17.82 | 26.71        |
| RNNsearch-50  | 26.75 | 34.16        |
| RNNsearch-50★ | 28.45 | 36.15        |
| Moses         | 33.30 | 35.63        |

RNNsearch-50★는 validation 성능이 더 이상 개선되지 않을 때까지 RNNsearch-50을 더 오래 학습한 모델이다.

전체 문장을 기준으로 Moses가 33.30으로 가장 높은 점수를 기록하였다. 그러나 더 오래 학습한 RNNsearch-50★도 28.45를 기록해 기존 RNNencdec-50의 17.82보다 크게 향상되었다. [UNK]가 없는 문장에서는 RNNsearch-50★가 36.15를 기록해 Moses의 35.63과 비슷하거나 조금 높은 성능을 보였다.

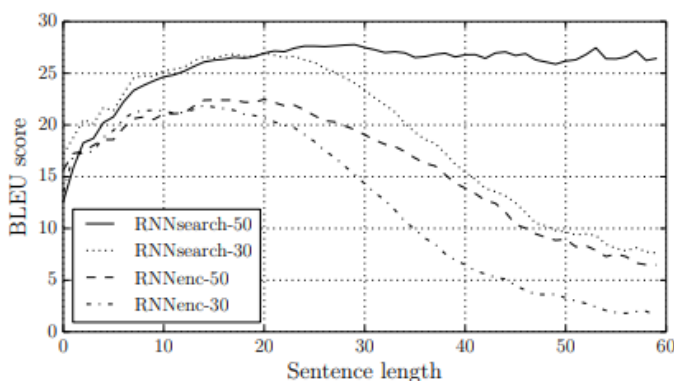
특히 Moses가 병렬 말뭉치 외에도 약 4억 1,800만 단어 규모의 별도 단일언어 말뭉치를 사용했다는 점을 고려하면, 하나의 end-to-end 신경망으로 이에 근접한 성능을 달성했다는 데 의미가 있다. 다만 전체 문장에서는 Moses의 성능이 여전히 더 높으므로, 제안 모델이 모든 조건에서 phrase-based 시스템을 능가했다고 해석해서는 안 된다.

## 문장 길이에 따른 성능

Attention의 효과는 긴 문장에서 더욱 뚜렷하게 나타났다. 기존 RNNencdec는 입력 문장의 길이가 증가하면서 BLEU score가 급격히 하락하였다. 반면 RNNsearch는 문장이 길어져도 성능을 비교적 안정적으로 유지하였다.

특히 RNNsearch-50은 길이가 50단어 이상인 문장에서도 뚜렷한 성능 저하를 보이지 않았다. 최대 30단어의 문장으로 학습한 RNNsearch-30도 최대 50단어의 문장으로 학습한 RNNencdec-50보다 높은 성능을 나타냈다.

이 결과는 성능 향상이 단순히 더 긴 문장으로 학습했기 때문이 아니라, 입력 문장 전체를 하나의 고정 길이 벡터로 완벽하게 압축할 필요가 없는 Attention 구조 자체에서 비롯되었음을 보여준다.



**Figure 2.** 문장 길이에 따른 BLEU score 비교. 가로축은 입력 문장의 길이, 세로축은 BLEU score이다. RNNencdec 계열은 문장이 길어질수록 성능이 크게 감소하지만, RNNsearch 계열은 상대적으로 안정적인 성능을 유지한다.

저자들이 제시한 긴 문장의 번역 예시에서도 비슷한 차이가 나타났다. 기존 RNNencdec는 문장의 앞부분은 비교적 정확하게 번역하다가 일정 길이를 넘어가면 원문의 의미에서 벗어나거나 세부 내용을 누락하였다.

예를 들어 admitting privilege를 설명하는 긴 문장에서 RNNencdec-50은 "병원 의료 종사자로서의 지위에 근거하여"라는 의미의 표현을 "그의 건강 상태에 따라"에 가까운 의미로 잘못 번역하였다. 반면 RNNsearch-50은 해당 부분의 의미를 비교적 정확히 보존하였다. 이는 Attention 기반 모델이 긴 문장에서 뒤쪽 정보까지 더 안정적으로 활용할 수 있음을 보여준다.

## Soft alignment의 시각화

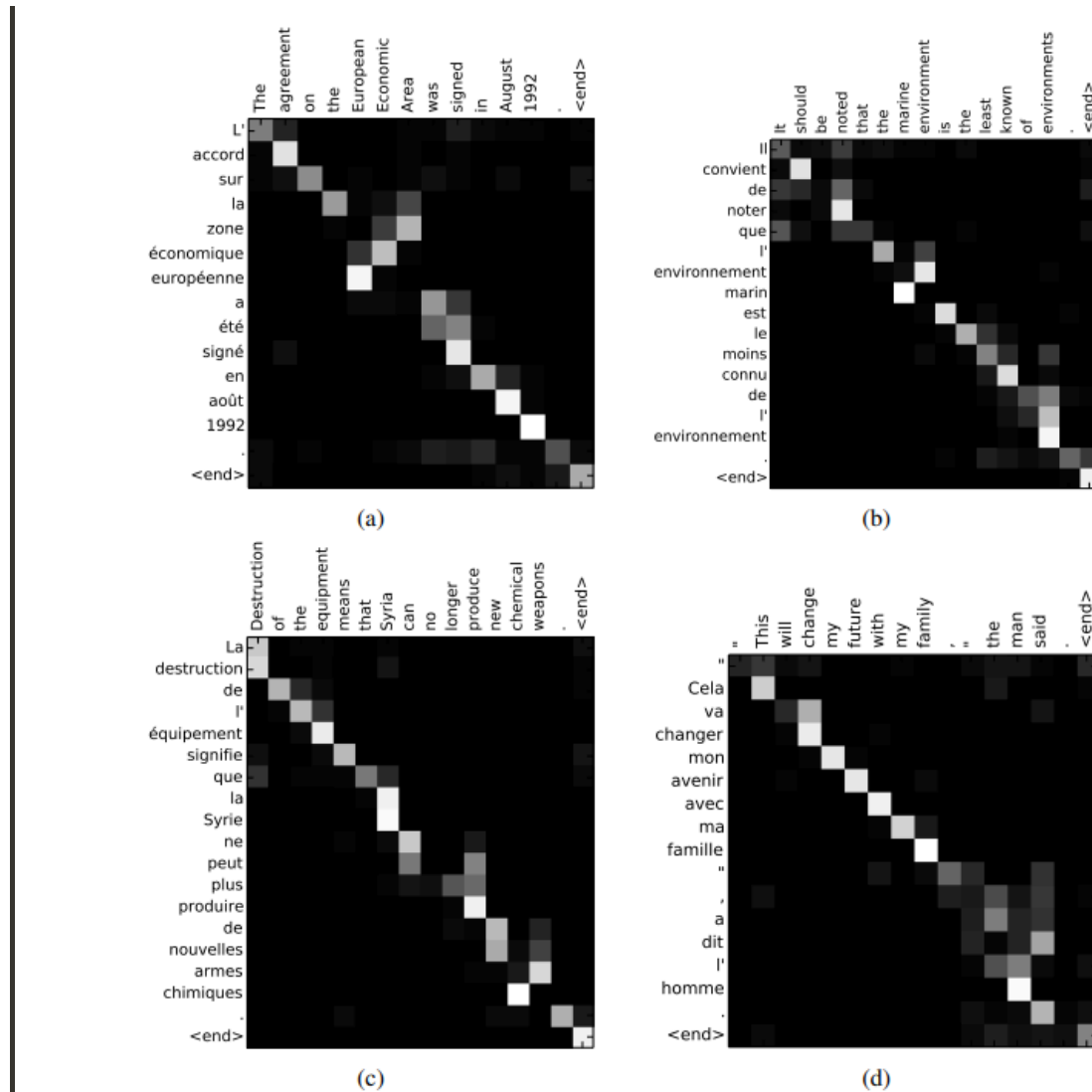
Attention weight  $\alpha_{ij}$ 를 행렬로 나타내면 번역문의 각 단어가 원문의 어느 위치를 중요하게 참고했는지 확인할 수 있다.

Figure 3에서 가로축은 영어 원문의 단어, 세로축은 모델이 생성한 프랑스어 번역문의 단어를 나타낸다. 밝은 칸일수록 해당 출력 단어를 생성할 때 해당 입력 단어에 높은 Attention weight를 부여했다는 의미이다.

영어와 프랑스어의 단어 순서가 대체로 유사하기 때문에 밝은 영역은 주로 대각선 방향으로 나타난다. 그러나 두 언어의 어순이 달라지는 부분에서는 대각선에서 벗어난 non-monotonic alignment도 관찰된다.

대표적으로 영어의 European Economic Area는 프랑스어에서 zone économique européenne처럼 명사와 수식어의 순서가 달라진다. 모델은 프랑스어의 zone을 영어의 Area와 연결하면서中间的 European과 Economic을 건너뛴 뒤, 앞쪽 단어를 하나씩 뒤집으며 전체 구를 완성하였다.

또한 the man을 l'homme으로 번역하는 예시에서는 the 하나만 보는 것이 아니라 the와 man을 함께 참고하였다. 프랑스어 관사의 형태는 뒤따르는 명사에 따라 le, la, les, l' 중 무엇이 될지 달라지므로, 관사를 번역하려면 뒤의 명사 정보도 함께 고려해야 한다. 여러 입력 위치를 동시에 참고하는 soft alignment가 hard alignment보다 유연하게 작동한 사례이다.



**Figure 3.** RNNsearch-50이 학습한 soft alignment 예시. 각 행은 번역문의 단어, 각 열은 원문의 단어에 대응한다. 밝은 영역은 해당 번역 단어를 생성할 때 높은 Attention weight가 부여된 원문 위치를 의미한다. 대각선 패턴은 두 언어에서 유사한 단어 순서를, 대각선에서 벗어난 패턴은 언어 간 어순 차이에 따른 non-monotonic alignment를 보여준다.

## 관련 연구와 차별점

Graves (2013)는 handwriting synthesis 과제에서 출력 기호와 입력 기호 사이의 정렬을 학습하는 유사한 접근을 제안하였다. 다만 해당 방법에서는 정렬 위치가 단조적으로 증가하도록 제한되어 가중치의 중심이 한 방향으로만 이동할 수 있었다.

기계번역에서는 언어 사이의 어순 차이로 인해 장거리 순서 재배열이 필요할 수 있다. 따라서 정렬 위치가 한 방향으로만 이동한다는 제약은 번역 문제에서 심각한 한계가 될 수 있다. 본 논문의 Attention은 모든 입력 위치에 자유롭게 가중치를 부여하므로 European Economic Area의 번역 사례와 같은 non-monotonic alignment를 학습할 수 있다.

또한 당시 많은 연구에서는 신경망을 phrase pair의 점수를 계산하거나 번역 후보를 재순위화하는 등 기존 통계적 기계번역 시스템의 하위 구성 요소로 활용하였다. 이에 비해 본 논문의 Neural Machine Translation은 원문으로부터 번역문을 직접 생성하는 독립적인 end-to-end 시스템을 지향한다.

## 한계점

첫째, 출력 단어를 생성할 때마다 모든 입력 위치에 대해 alignment score를 계산해야 한다. 입력 문장의 길이를  $T_x$ , 출력 문장의 길이를  $T_y$ 라고 하면 문장 쌍마다 alignment model을 약  $T_x T_y$ 번 평가해야 한다.

논문에서는 일반적인 번역 문장의 길이가 대체로 15~40단어 수준이므로 이 계산 비용이 기계번역에서는 치명적이지 않다고 설명한다. 그러나 더 긴 sequence를 다루는 다른 과제에서는 적용을 제한하는 요인이 될 수 있다.

둘째, Attention은 fixed-length bottleneck을 완화했지만 RNN 자체의 제약을 없앤 것은 아니다. Encoder와 Decoder가 모두 RNN이므로 이전 시점의 계산이 끝나야 다음 시점을 계산할 수 있고, sequence 내의 여러 위치를 동시에 처리하기는 어렵다.

셋째, 각 언어의 vocabulary를 빈도가 높은 30,000개 단어로 제한하고 나머지 단어를 [UNK]로 처리하였다. 실제 실험에서도 [UNK]가 포함된 전체 문장과 [UNK]가 없는 문장 사이에 큰 BLEU score 차이가 나타난다. 저자들 역시 unknown word와 rare word를 더 잘 처리하는 방법을 향후 과제로 제시한다.

마지막으로 실험이 영어-프랑스어 번역에 집중되어 있다. 따라서 어순이나 문법 구조가 크게 다른 모든 언어쌍에서도 동일한 수준의 성능 향상이 나타난다고 단정할 수는 없다.

## 결론

본 논문은 기존 RNN Encoder-Decoder가 입력 문장 전체를 하나의 고정 길이 context vector에 압축하면서 발생하는 정보 병목을 문제로 제기하였다. 이를 해결하기 위해 입력 문장의 각 위치를 Bidirectional RNN의 annotation으로 보존하고, Decoder가 번역 단어를 생성할 때마다 각 annotation의 중요도를 계산하는 Attention mechanism을 제안하였다.

Alignment score를 softmax로 정규화한 Attention weight는 현재 출력 단어와 입력 위치 사이의 soft alignment 역할을 한다.

Decoder는 이 가중치를 이용해 출력 시점마다 서로 다른 context vector를 구성한다. 따라서 모델은 문장 전체를 하나의 벡터에 완벽하게 저장할 필요 없이, 필요한 시점에 관련된 입력 정보를 선택적으로 가져올 수 있다.

실험 결과 RNNsearch는 모든 조건에서 기존 RNNencdec보다 높은 BLEU score를 기록했으며, 특히 긴 문장에서 성능 차이가 크게 나타났다. Attention weight를 시각화한 결과에서도 영어와 프랑스어 사이의 대체로 단조로운 정렬뿐 아니라, 명사와 수식어의 순서가 달라지는 비단조적 정렬까지 학습한 모습을 확인할 수 있었다.

무엇보다 alignment mechanism을 포함한 번역 시스템의 모든 구성 요소가 올바른 번역문의 log-probability를 높이는 하나의 목적 함수 아래에서 함께 학습된다는 점이 기존의 분리된 기계번역 시스템과 구별되는 특징이다.

본 논문의 가장 큰 의의는 단순히 BLEU score를 개선한 데 있지 않다. 입력 문장의 정보를 하나의 고정된 표현에 모두 압축하는 대신, 현재 출력에 필요한 정보를 매 시점 동적으로 선택하도록 정보 전달 방식을 바꾸었다는 점이 핵심이다. 기존 구조의 병목을 명확히 분석하고, 정렬과 번역을 하나의 미분 가능한 모델 안에서 공동 학습하는 직관적인 해결책을 제시했다는 점에서 의미 있는 연구라고 평가할 수 있다.